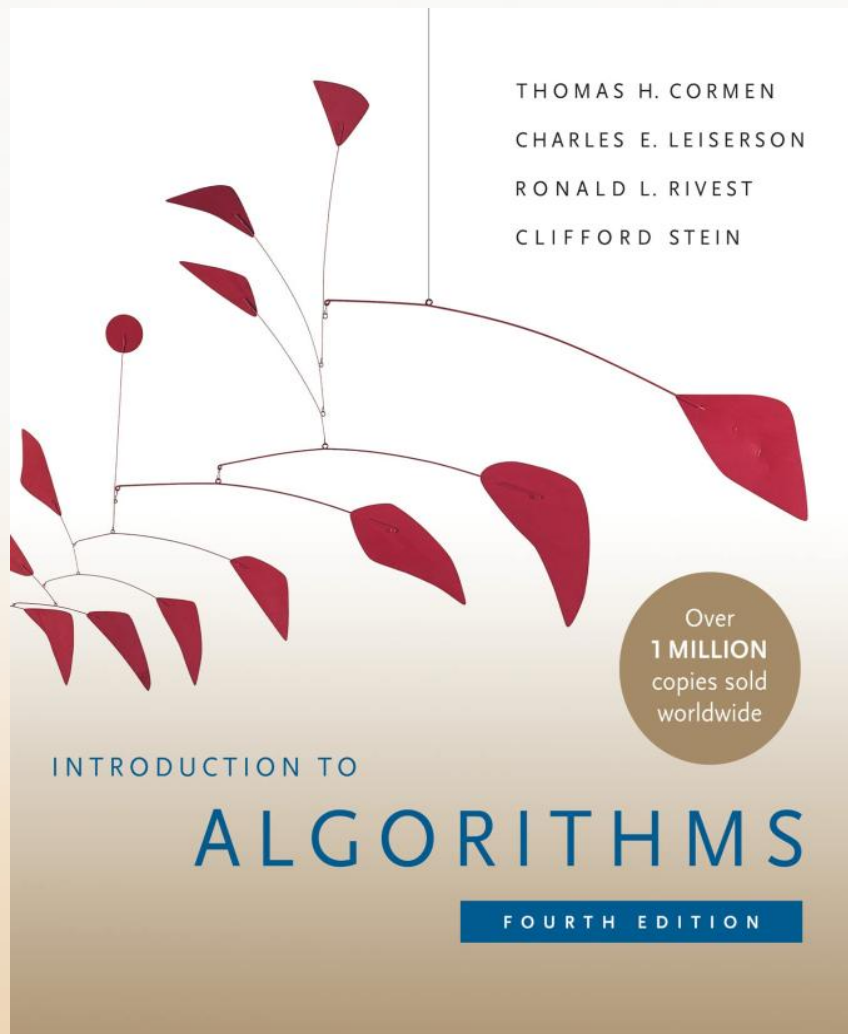


بہ نام خدا طراحی الگوریتم

مقدمه ایی بر الگوریتم ها

منابع

۱- مقدمه ای بر الگوریتم ها
ویرایش چهارم، کرمن، ۲۰۲۲



Cormen, Thomas H.; Leiserson, Charles E.; Rivest, Ronald L. (1990). *Introduction to Algorithms* (1st ed.). MIT Press and McGraw-Hill. ISBN 0-262-03141-8. •

Cormen, Thomas H.; Leiserson, Charles E.; Rivest, Ronald L.; Stein, Clifford (2001) •
[1990]. *Introduction to Algorithms* (2nd ed.). MIT Press and McGraw-Hill. ISBN 0-262-03293-7. 12 printings up to 2009, errata:^[11]

Cormen, Thomas H.; Leiserson, Charles E.; Rivest, Ronald L.; Stein, Clifford (2009) •
[1990]. *Introduction to Algorithms* (3rd ed.). MIT Press and McGraw-Hill. ISBN 0-262-03384-4. 1320 pp., 5 printings up to 2016, errata:^[12]

Cormen, Thomas H.; Leiserson, Charles E.; Rivest, Ronald L.; Stein, Clifford (2022) •
[1990]. *Introduction to Algorithms* (4th ed.). MIT Press and McGraw-Hill. ISBN 0-262-04630-X. 1312 pp., errata:

اهداف درس

- آشنایی با روش های تحلیل عملکرد الگوریتم ها
- آشنایی با الگوها و روش های مهم طراحی الگوریتم مانند:
 - تقسیم و غلبه
 - برنامه نویسی پویا
 - الگوریتم های حریصانه
 - گراف

تعریف الگوریتم

- الگوریتم یک روش محاسباتی کاملاً مشخص است که مقدار یا مجموعه ای از مقادیر را به عنوان ورودی دریافت می کند و مقدار یا مجموعه ای از مقادیر را به عنوان خروجی تولید کند



مثال: مسئله جستجو

- ورودی :
 - دنباله ای از n عدد $(a_1, a_2, \dots, a_n)$
 - عنصری که به دنبال آن هستیم x
- خروجی :
 - نمایه عنصر x در صورت وجود

جستجوی دودوی

```
FUNCTION BinarySearchIterative(A, target)
  # Precondition: A is sorted in ascending order
  low ← 0
  high ← length(A) - 1

  WHILE low ≤ high DO
    mid ← low + (high - low) // 2    # avoids overflow
    IF A[mid] = target THEN
      RETURN mid                    # found
    ELSE IF A[mid] < target THEN
      low ← mid + 1                  # search right half
    ELSE
      high ← mid - 1                 # search left half
    END IF
  END WHILE

  RETURN -1                          # not found
END FUNCTION
```

جستجوی دودوی (بازگشتی)

```
FUNCTION BinarySearchRecursive(A, target, low, high)
    # Precondition: A is sorted in ascending order
    IF low > high THEN
        RETURN -1                # not found
    END IF

    mid ← low + (high - low) // 2
    IF A[mid] = target THEN
        RETURN mid
    ELSE IF A[mid] < target THEN
        RETURN BinarySearchRecursive(A, target, mid + 1, high)
    ELSE
        RETURN BinarySearchRecursive(A, target, low, mid - 1)
    END IF
END FUNCTION

# wrapper to call recursive version:
FUNCTION BinarySearch(A, target)
    RETURN BinarySearchRecursive(A, target, 0, length(A) - 1)
END FUNCTION
```


مثال: مساله مرتب سازی

- ورودی: دنباله ای از n عدد $(a_1, a_2, \dots, a_n)$
خروجی: یک جایگشت (چینش) $(a'_1, a'_2, \dots, a'_n)$ از دنباله ورودی به طوری که $(a'_1 < a'_2 < \dots < a'_n)$

دنباله ورودی: 31, 41, 59, 26, 41, 58

دنباله خروجی: 26, 31, 41, 41, 58, 59

الگوریتم صحیح

- یک الگوریتم درست است اگر برای هر نمونه ورودی، با خروجی صحیح متوقف شود.
- یک الگوریتم نادرست ممکن است در بعضی موارد ورودی اصلاً متوقف نشود، یا ممکن است با پاسخی غیر از جواب مورد نظر متوقف شود.
- الگوریتم های نادرست گاهی اوقات می توانند مفید باشند اگر میزان خطای آنها قابل کنترل باشد.

انتخاب الگوریتم

- دنباله فیبوناتچی:

$F(0) = 0$, $F(1) = 1$, and

$F(n) = F(n-1) + F(n-2)$ for $n > 1$.

الگوریتم های زیادی وجود دارد، اما کارآمدترین کدام است؟

```
function fib1(n){  
    if n < 2 then return n;  
    else return fib1(n-1) + fib1(n-2);  
}
```

```
function fib2(n){  
    i = 1; j = 0;  
    for k = 1 to n do { j = i+j; i = j- i;}  
    return j;  
}
```

مقایسه مرتبه زمانی

n	10	20	30	50	100	10000	1000 000	1000000 00
fib1	8 ms	1 s	2 min	21 days				
fib2	1/6 ms	1/3 ms	1/2 ms	3/4 ms	3/2 ms	150 ms	15 s	25 min

